Project Submission – Database Administration

Vehicle Service Management Database

Student Details

STUDENT NAME	CHANDRU MP
Register Number	2428C0371
Department	ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
ID	98bc7bd2397c4bc8d9945b4f39d565b8@vtpt2026.in
College	AKSHAYA COLLEGE OF ARTS AND SCIENCE
Date of Submission	25.09.2026

1. Problem Statement & Objectives

Problem Statement

The vehicle service center provides maintenance and repair services for customers and their vehicles. The center needs to maintain customer details, vehicle information, and records of services performed on each vehicle.

Managing these details manually can make it difficult to track service history, find customer or vehicle information, and calculate service costs accurately. The proposed ***Vehicle Service Management Database*** *solves these problems by storing the information in related tables and allowing the service center to efficiently add, update, delete, search, and retrieve records using SQL queries.*

Objectives

The main objectives of the CareTrack database are:

- To design a relational database for managing clinic operations.
- To maintain patient and doctor information in organized tables.
- To record and monitor patient appointments.
- To maintain treatment and consultation records.

- To establish relationships between patients, doctors, appointments, and treatments.
- To implement primary keys, foreign keys, and integrity constraints.
- To perform CRUD operations using SQL.
- To retrieve information using filtering, sorting, grouping, and joins.
- To calculate treatment-related information using aggregate functions.
- To demonstrate the practical application of relational database concepts using MySQL Workbench

2. Database Design

2.1 Database Structure

The Customer and Vehicle Management Database is a relational database developed to maintain customer information and the vehicles owned by them. The database contains two main tables: Customer and Vehicle. These tables are connected using primary and foreign keys to maintain a proper relationship between customers and their vehicles.

Tables Used

Table Name	Purpose
Customer	Stores customer information such as Customer ID, customer name, contact number, email, and address.
Vehicle	Stores vehicle information such as Vehicle ID, vehicle number, vehicle type, vehicle model, and the associated Customer ID.
Service	Stores vehicle service records such as Service ID, Vehicle ID, service date, service type, service status, and service cost.

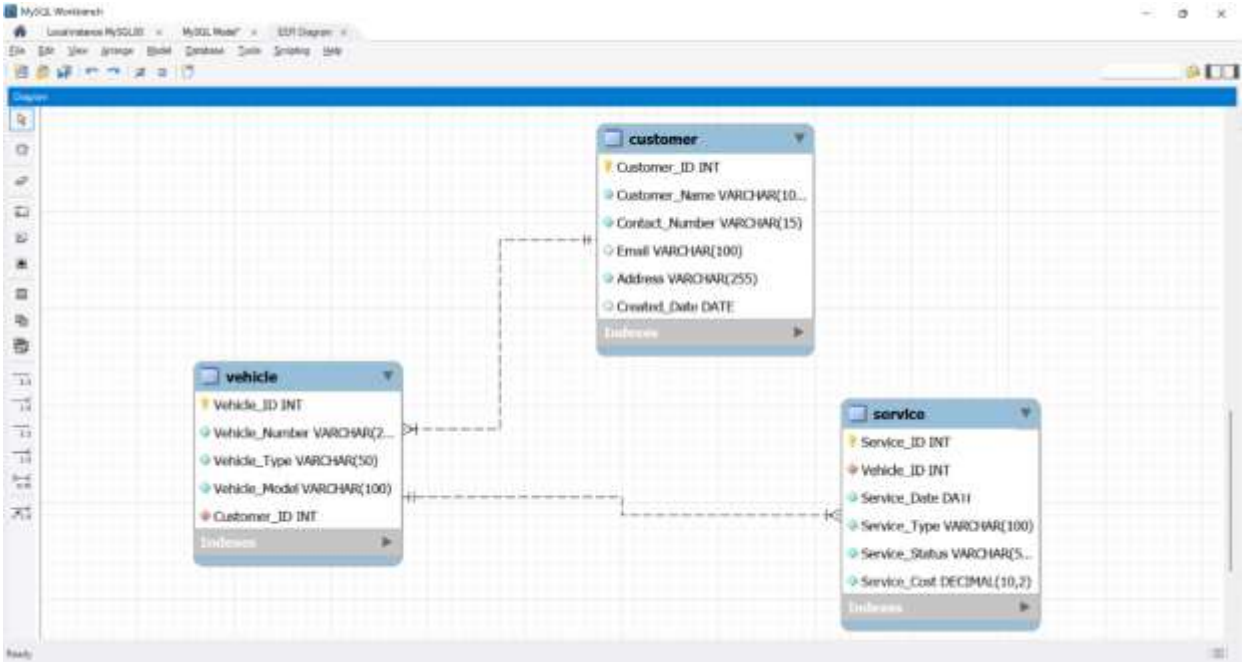
Relationships:

- **Customer → Vehicle:** One customer can have multiple vehicles.
- **Vehicle → Service:** One vehicle can have multiple service records.

2.2 Entity Relationship Diagram

The Entity Relationship Diagram (ERD) represents the structure of the Vehicle Service Management Database. It contains three main entities: Customer, Vehicle, and Service. A customer can have multiple vehicles, and each vehicle can have multiple service records. Primary keys and foreign keys are used to establish the relationships and maintain data integrity.

Diagram:

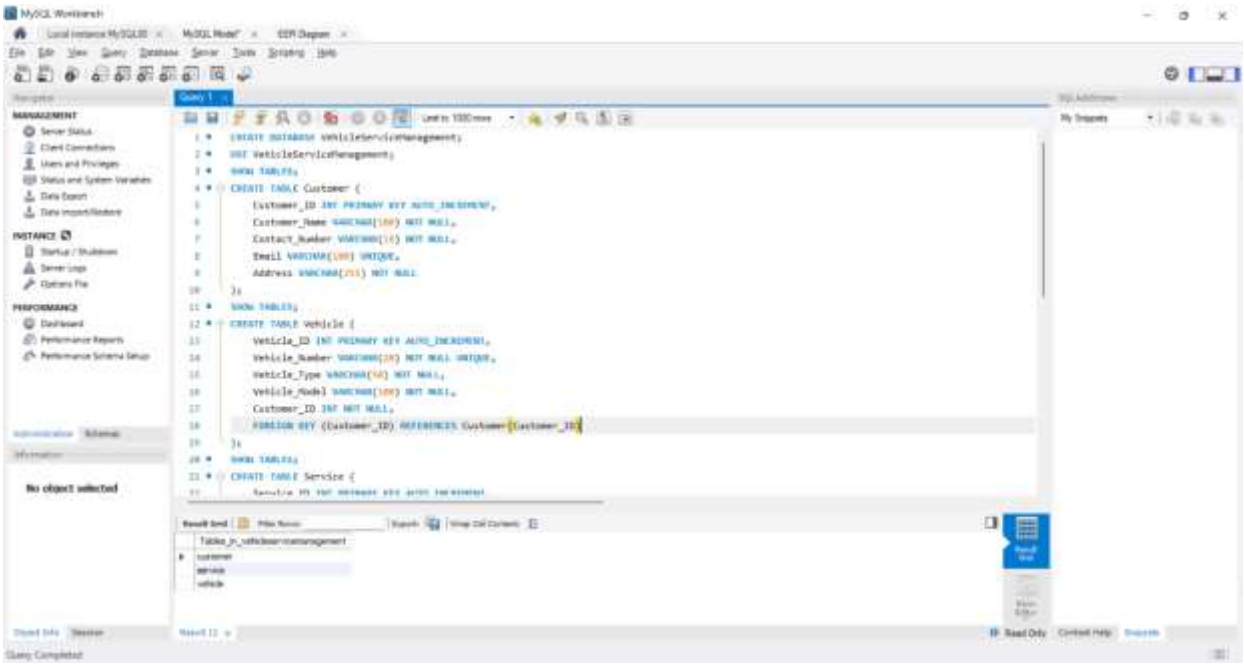


2.3 Technologies Used

Technology / Tool	Purpose
MySQL	Database management system used to store and manage customer, vehicle, and service data.
MySQL Workbench	Used for database design, table creation, relationship visualization, and SQL query execution.
SQL	Used for database creation, table management, data manipulation, filtering, aggregation, grouping, and retrieving information.

3. Practical Implementation

3.1 Database and Table Creation



Database Creation

```
CREATE DATABASE VehicleServiceManagement;

USE VehicleServiceManagement;
```

Customer Table

```
CREATE TABLE Customer (

      Customer_ID INT PRIMARY KEY AUTO_INCREMENT,

      Customer_Name VARCHAR(100) NOT NULL,

      Contact_Number VARCHAR(15) NOT NULL,

      Email VARCHAR(100) UNIQUE,
```

```
    Address VARCHAR(255) NOT NULL
);
```

Vehicle Table

```
CREATE TABLE Vehicle (
    Vehicle_ID INT PRIMARY KEY AUTO_INCREMENT,
    Vehicle_Number VARCHAR(20) NOT NULL UNIQUE,
    Vehicle_Type VARCHAR(50) NOT NULL,
    Vehicle_Model VARCHAR(100) NOT NULL,
    Customer_ID INT NOT NULL,
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
);
```

Service Table

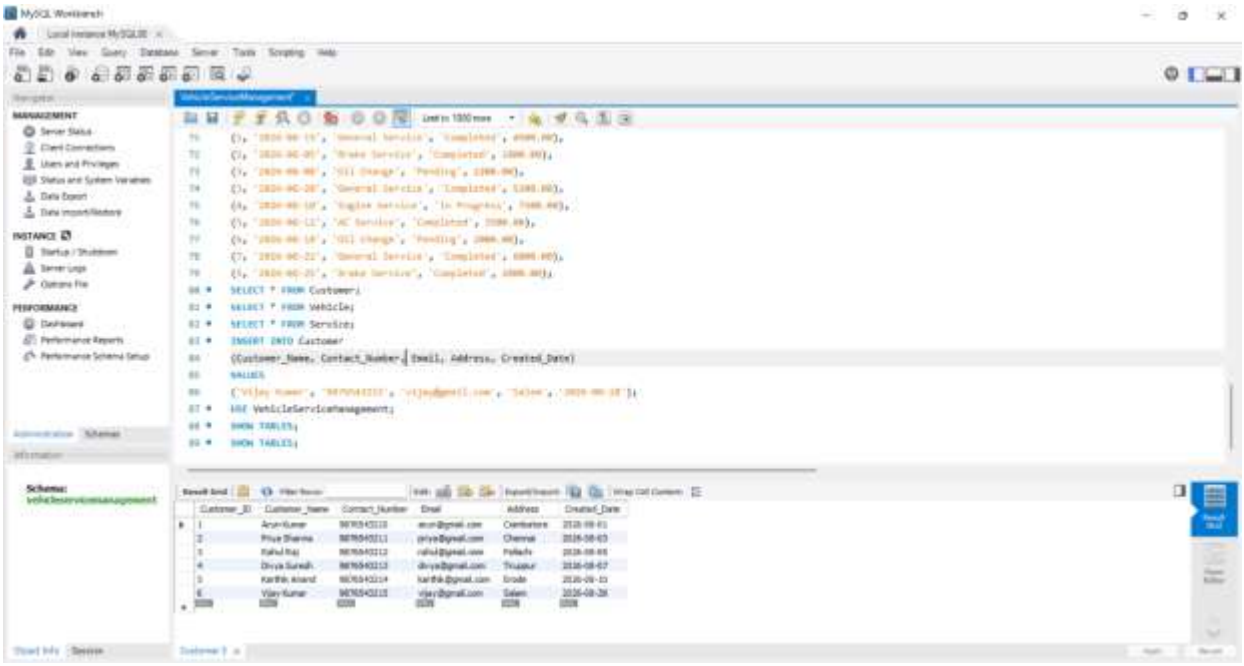
```
CREATE TABLE Service (
    Service_ID INT PRIMARY KEY AUTO_INCREMENT,
    Vehicle_ID INT NOT NULL,
    Service_Date DATE NOT NULL,
    Service_Type VARCHAR(100) NOT NULL,
    Service_Status VARCHAR(50) NOT NULL,
    Service_Cost DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (Vehicle_ID) REFERENCES Vehicle(Vehicle_ID)
);
```

3.2 Constraints and Relationships

Primary keys uniquely identify each record, while foreign keys establish relationships between the tables. ***NOT NULL*** *ensures required information is provided, and* ***UNIQUE*** *prevents duplicate values.*

Table	Primary Key	Foreign Key	Other Constraints
Customer	Customer_ID	None	Customer_Name, Contact_Number, and Address are NOT NULL; Email is UNIQUE; Customer_ID is AUTO_INCREMENT
Vehicle	Vehicle_ID	Customer_ID → Customer(Customer_ID)	Vehicle_Number is NOT NULL and UNIQUE; Vehicle_Type, Vehicle_Model, and Customer_ID are NOT NULL; Vehicle_ID is AUTO_INCREMENT
Service	Service_ID	Vehicle_ID → Vehicle(Vehicle_ID)	Service_Date, Service_Type, Service_Status, and Service_Cost are NOT NULL; Service_ID is AUTO_INCREMENT

3.3 Data Insertion



Customer Data

```
INSERT INTO Customer

(Customer_Name, Contact_Number, Email, Address, Created_Date)

VALUES

('Arun Kumar', '9876543210', 'arun@gmail.com', 'Coimbatore', '2026-08-01'),

('Priya Sharma', '9876543211', 'priya@gmail.com', 'Chennai', '2026-08-03'),

('Rahul Raj', '9876543212', 'rahul@gmail.com', 'Pollachi', '2026-08-05'),

('Divya Suresh', '9876543213', 'divya@gmail.com', 'Tiruppur', '2026-08-07'),

('Karthik Anand', '9876543214', 'karthik@gmail.com', 'Erode', '2026-08-10');
```

Vehicle Data

```
INSERT INTO Vehicle

(Vehicle_Number, Vehicle_Type, Vehicle_Model, Customer_ID)

VALUES

('TN37AB1234', 'Car', 'Hyundai i20', 1),

('TN38CD5678', 'Bike', 'Honda Activa', 1),

('TN01EF9012', 'Car', 'Maruti Swift', 2),

('TN66GH3456', 'Bike', 'Royal Enfield Classic 350', 3),

('TN39IJ7890', 'Car', 'Tata Nexon', 4),

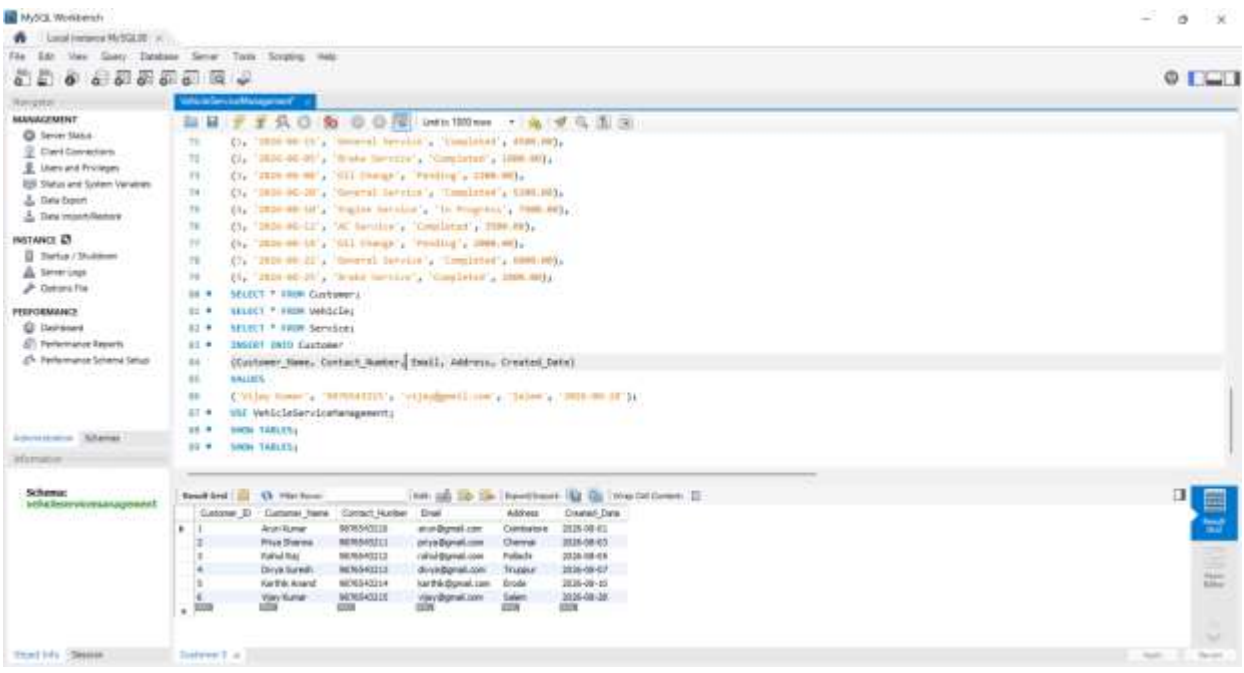
('TN40KL2345', 'Bike', 'Yamaha FZ', 4),
```

```
('TN33MN6789', 'Car', 'Toyota Glanza', 5);
```

Service Data

```
INSERT INTO Service
(Vehicle_ID, Service_Date, Service_Type, Service_Status, Service_Cost)
VALUES
(1, '2026-08-02', 'Oil Change', 'Completed', 2500.00),
(1, '2026-08-15', 'General Service', 'Completed', 4500.00),
(2, '2026-08-05', 'Brake Service', 'Completed', 1800.00),
(3, '2026-08-08', 'Oil Change', 'Pending', 2200.00),
(3, '2026-08-20', 'General Service', 'Completed', 5200.00),
(4, '2026-08-10', 'Engine Service', 'In Progress', 7500.00),
(5, '2026-08-12', 'AC Service', 'Completed', 3500.00),
(6, '2026-08-18', 'Oil Change', 'Pending', 2000.00),
(7, '2026-08-22', 'General Service', 'Completed', 6000.00),
(5, '2026-08-25', 'Brake Service', 'Completed', 2800.00);
```

3.4 SQL Operations



INSERT

```
INSERT INTO Customer

(Customer_Name, Contact_Number, Email, Address, Created_Date)

VALUES

('Vijay Kumar', '9876543215', 'vijay@gmail.com', 'Salem', '2026-08-
28');
```

SELECT

```
SELECT *

FROM Customer;
```

UPDATE

```
UPDATE Customer

SET Contact_Number = '9876543299',

      Address = 'Namakkal'

WHERE Customer_ID = 6;
```

DELETE

```
DELETE FROM Customer

WHERE Customer_ID = 6;
```

WHERE

```
SELECT *

FROM Service

WHERE Service_Status = 'Completed';
```

ORDER BY

```
SELECT *
```

```
FROM Service

ORDER BY Service_Cost DESC;
```

LIKE

```
SELECT *

FROM Vehicle

WHERE Vehicle_Number LIKE 'TN37%';
```

BETWEEN

```
SELECT *

FROM Service

WHERE Service_Cost BETWEEN 2000 AND 5000;
```

Aggregate Functions

```
SELECT COUNT(*) AS Total_Service_Records

FROM Service;


SELECT SUM(Service_Cost) AS Total_Service_Cost

FROM Service;


SELECT AVG(Service_Cost) AS Average_Service_Cost

FROM Service;


SELECT MAX(Service_Cost) AS Highest_Service_Cost

FROM Service;a2
```

```
SELECT MIN(Service_Cost) AS Lowest_Service_Cost

FROM Service;
```

GROUP BY

```
SELECT

      Service_Type,

      COUNT(*) AS Number_of_Services,

      SUM(Service_Cost) AS Total_Cost

FROM Service

GROUP BY Service_Type;
```

INNER JOIN

```
SELECT

      Customer.Customer_Name,

      Vehicle.Vehicle_Number,

      Vehicle.Vehicle_Model,

      Service.Service_Date,

      Service.Service_Type,

      Service.Service_Status,

      Service.Service_Cost

FROM Customer

INNER JOIN Vehicle

      ON Customer.Customer_ID = Vehicle.Customer_ID

INNER JOIN Service
```

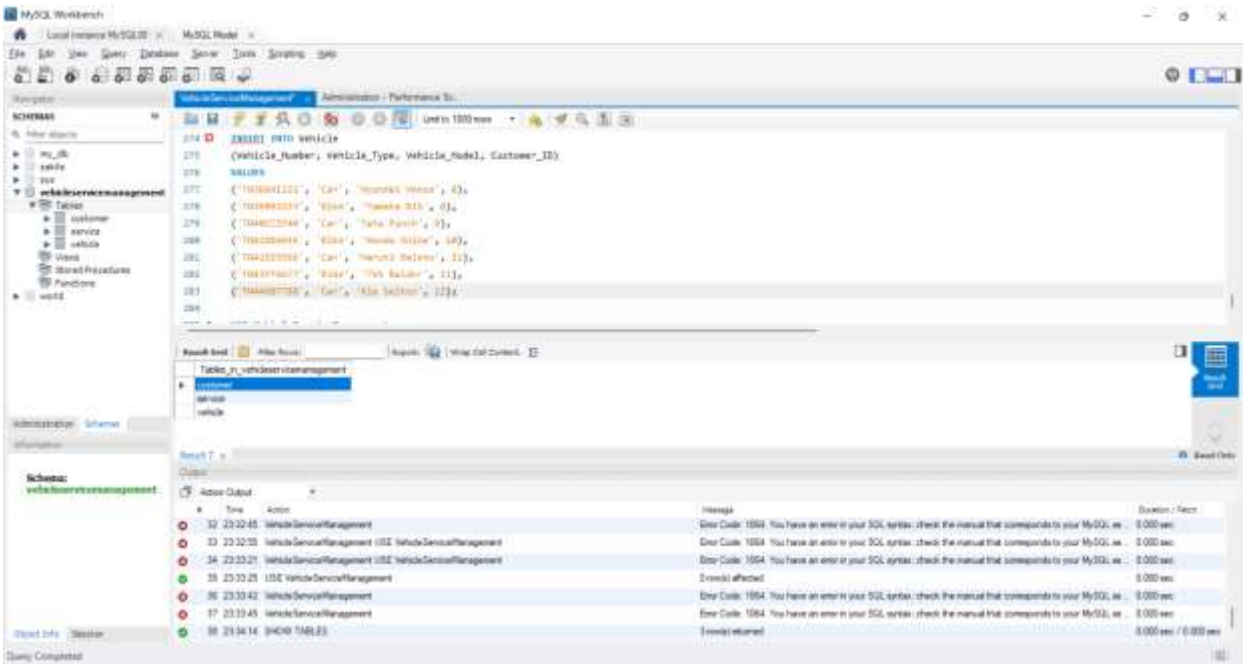
```
ON Vehicle.Vehicle_ID = Service.Vehicle_ID;
```

3.5 Project Execution

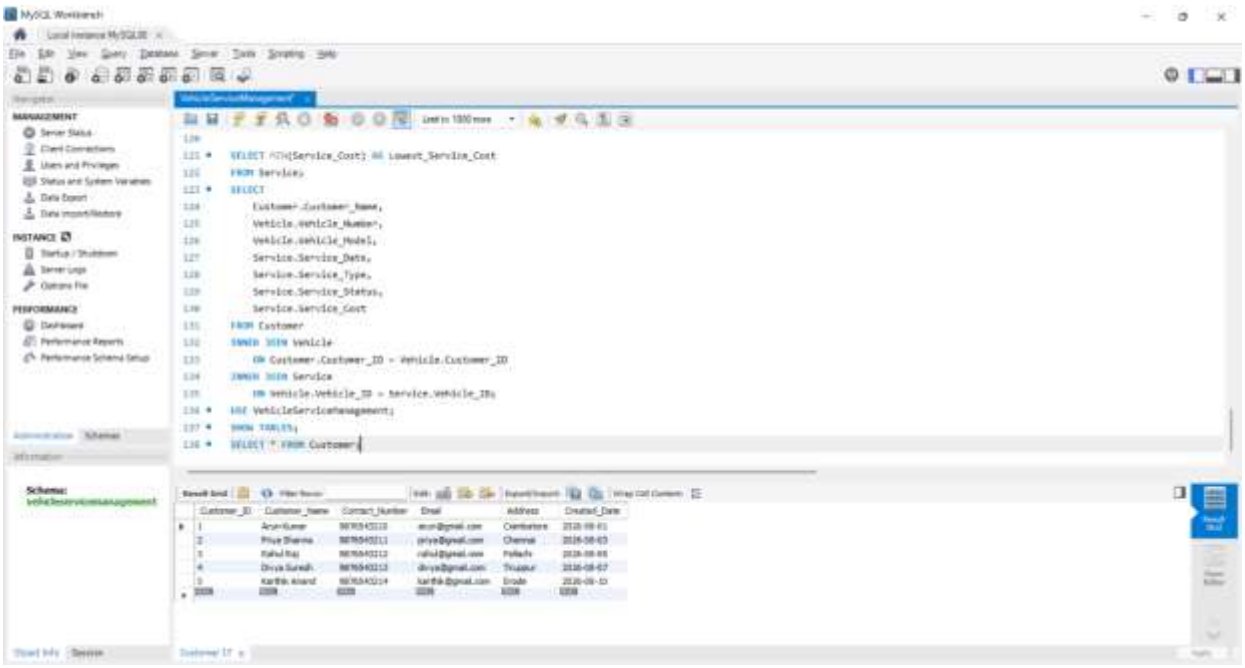
The Vehicle Service Management Database was successfully developed in MySQL Workbench using SQL commands. The Customer, Vehicle, and Service tables were created with appropriate primary keys, foreign keys, and constraints to maintain data integrity. Sample records were inserted and relationships were established between the tables. Various SQL operations, including INSERT, SELECT, UPDATE, DELETE, filtering, sorting, aggregate functions, grouping, and INNER JOIN, were performed using the MySQL Workbench SQL Editor. The outputs of the executed queries were verified through the result grid, demonstrating the successful implementation and management of the database.

4. Output Screenshots

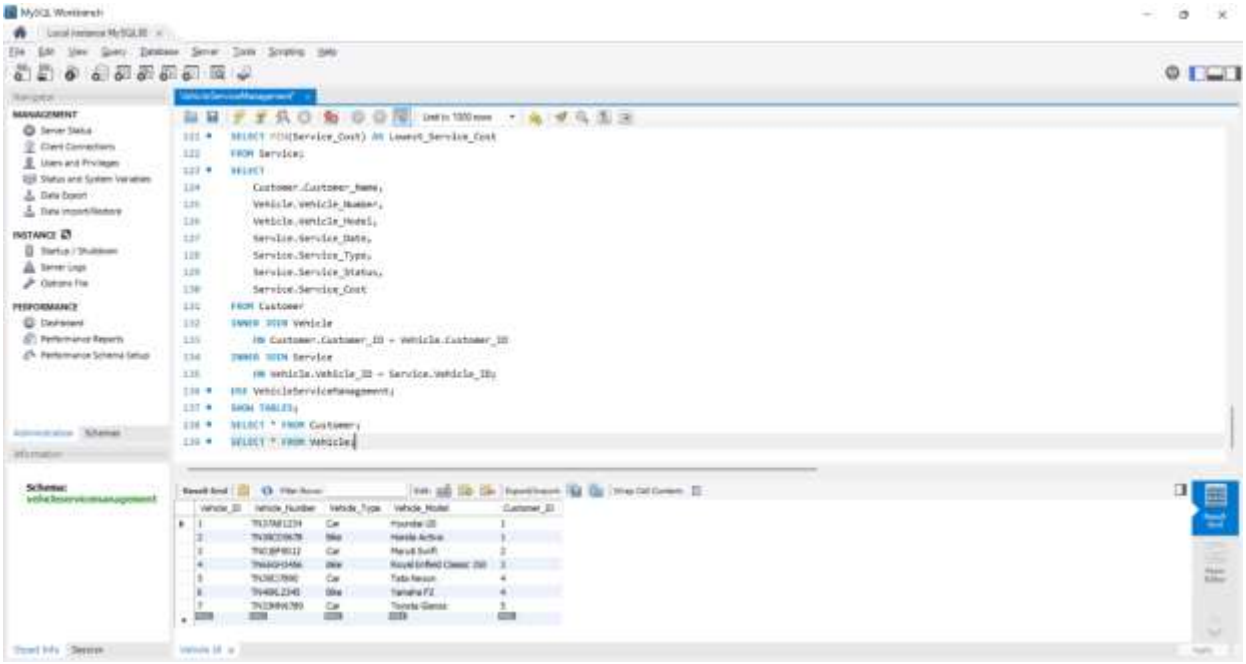
Screenshot 1 – Database and Tables



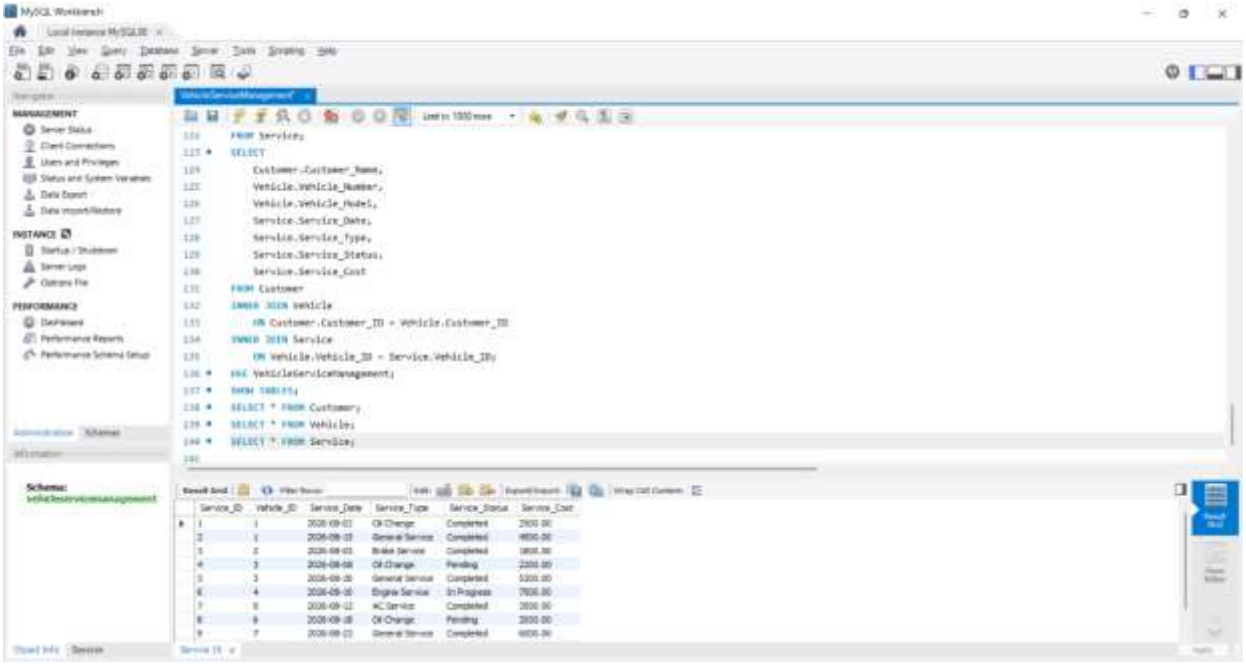
Screenshot 2 – Customer Table with Records



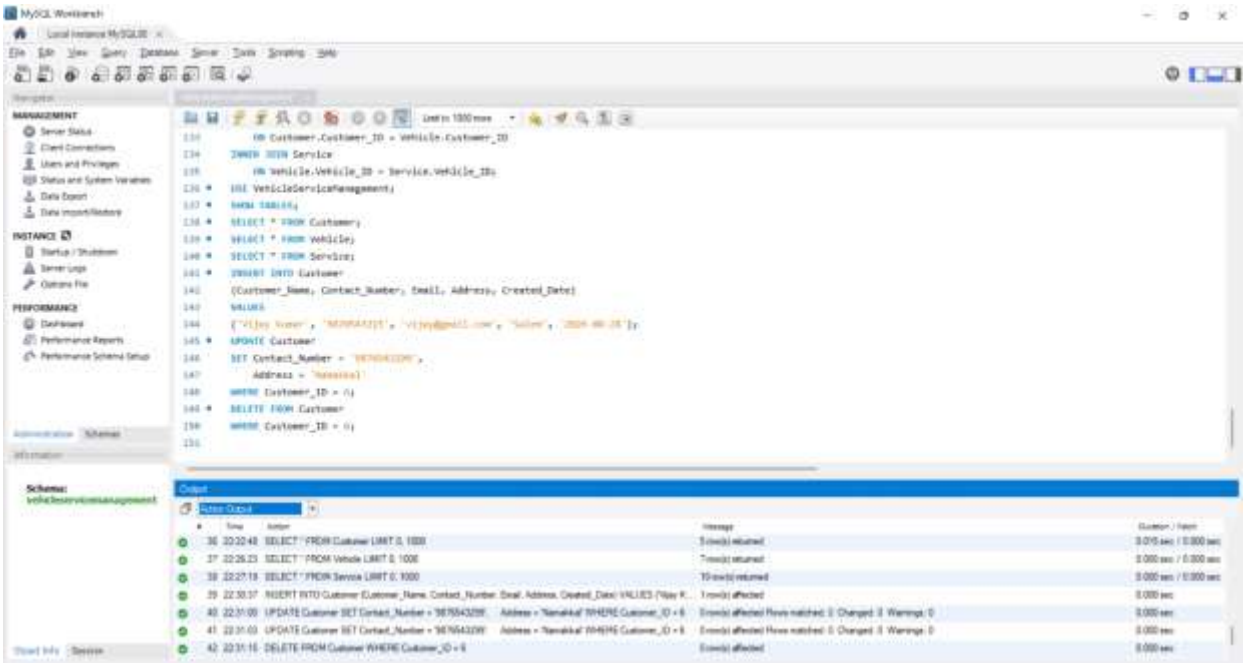
Screenshot 3 – Vehicle Table with Records



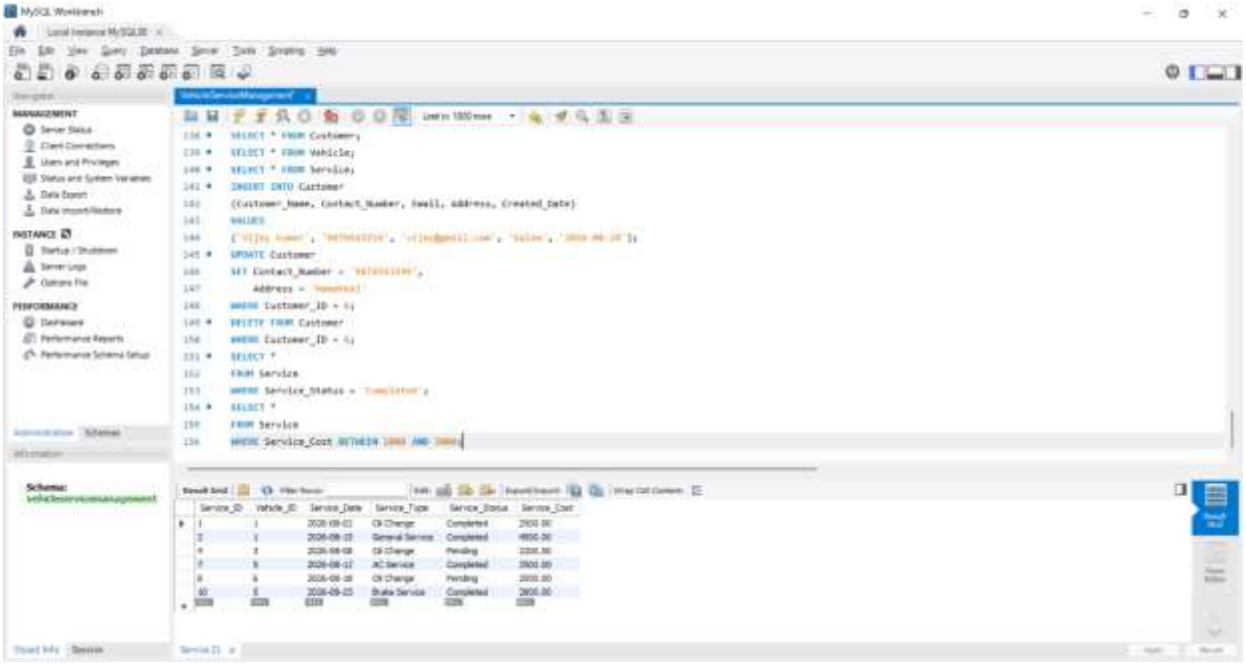
Screenshot 4 – Service Table with Records



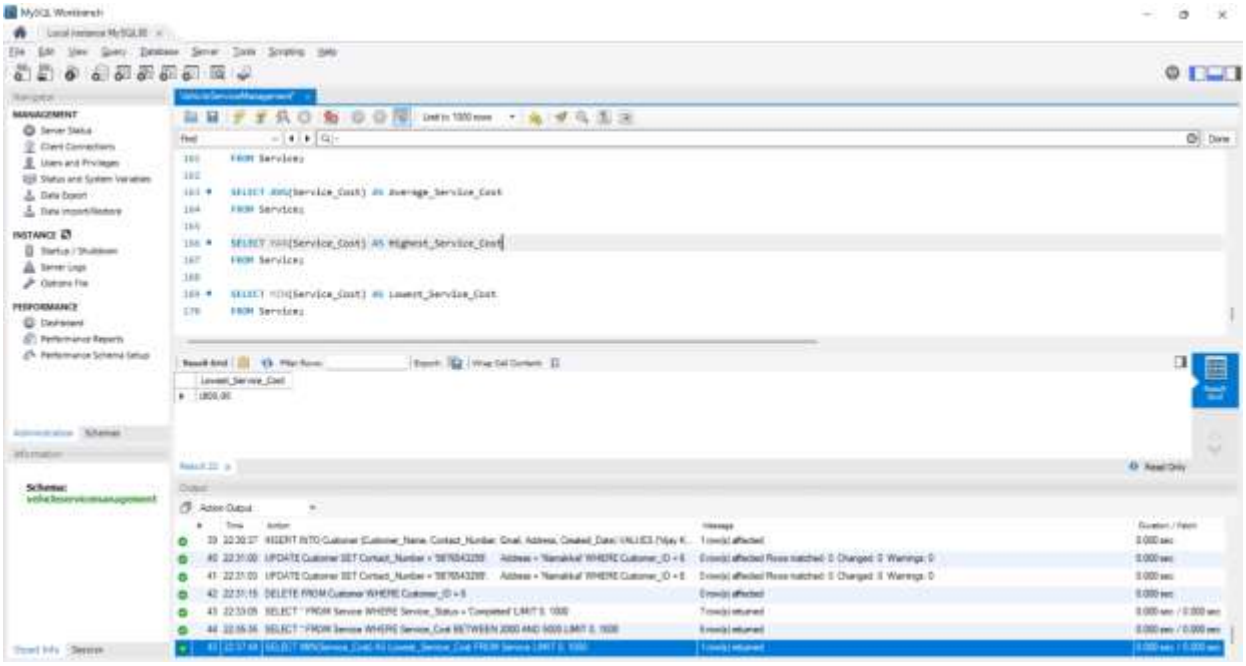
Screenshot 5 – INSERT / UPDATE / DELETE Operations



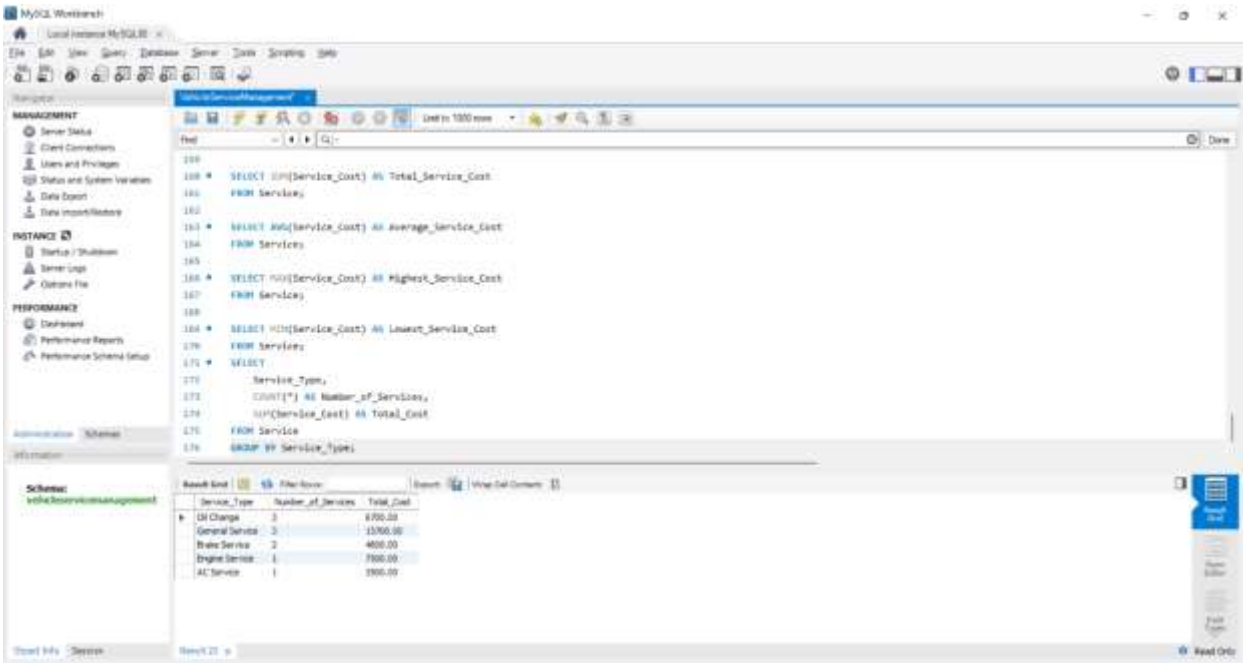
Screenshot 6 – SELECT and Filtering Operations



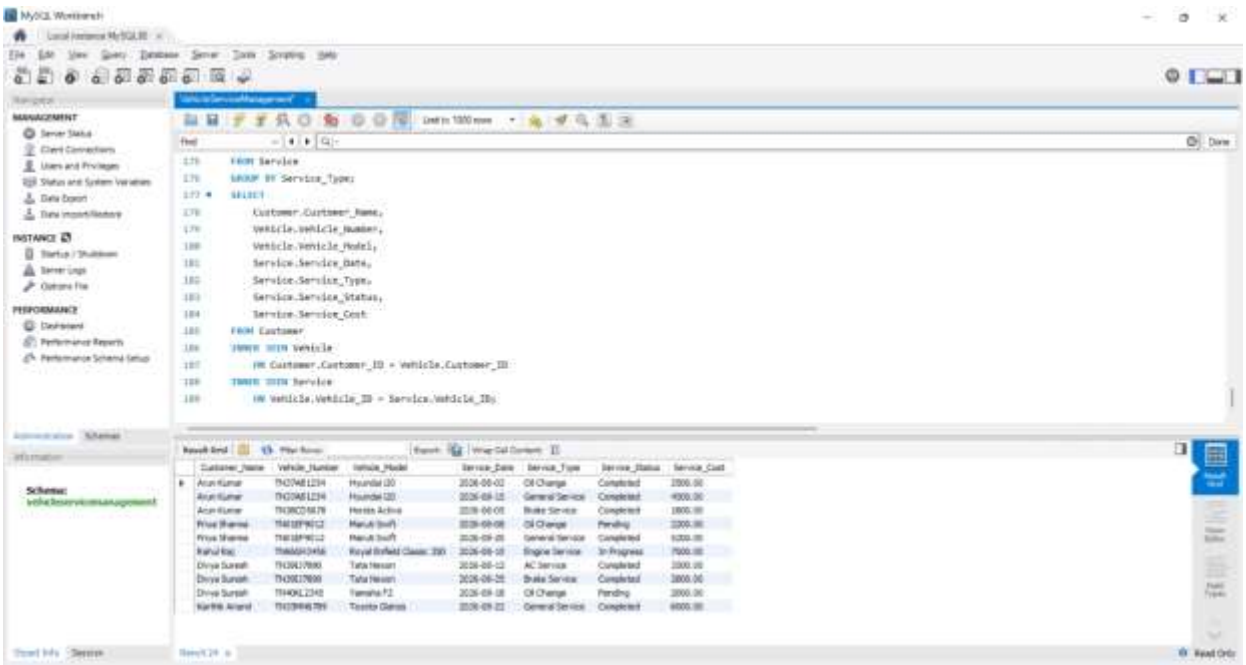
Screenshot 7 – Aggregate Functions



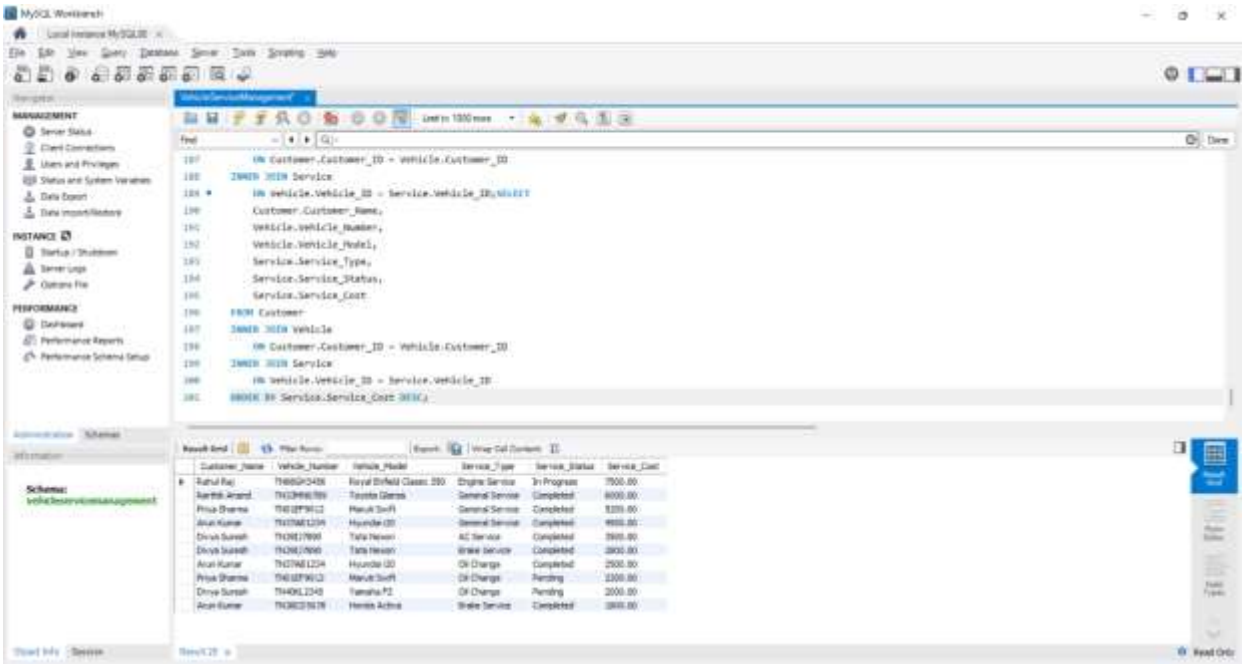
Screenshot 8 – GROUP BY Operation



Screenshot 9 – INNER JOIN Operation



Screenshot 10 – Final Database / Query Results



5. Technical Explanation

Database Design

*The **Vehicle Service Management Database** is designed using three separate tables: **Customer, Vehicle, and Service**. Each table stores information related to a specific entity.*

- ***Customer Table:***

  *Stores customer details such as **Customer ID, customer name, contact number, email, address, and created date**.*

- ***Vehicle Table:***

  *Stores vehicle details such as **Vehicle ID, vehicle number, vehicle type, vehicle model, and Customer ID** of the vehicle owner.*

- ***Service Table:***

  *Stores service details such as **Service ID, Vehicle ID, service date, service type, service status, and service cost**.*

*The tables are separated to reduce unnecessary duplication and maintain a proper **relational database structure**. A **customer can own multiple vehicles**, while a **vehicle can have multiple service records**. These relationships are maintained using **foreign keys**, connecting the Customer table with the Vehicle table and the Vehicle table with the Service table.*

Keys and Constraints

***Primary keys** are used to uniquely identify each record in the tables.*

- *Customer_ID is the primary key of the **Customer** table.*
- *Vehicle_ID is the primary key of the **Vehicle** table.*
- *Service_ID is the primary key of the **Service** table.*

***Foreign keys** are used to establish relationships between tables.*

- *Vehicle.Customer_ID references Customer.Customer_ID.*
- *Service.Vehicle_ID references Vehicle.Vehicle_ID.*

Other constraints were also used to maintain data integrity:

- ***NOT NULL** ensures that important fields cannot be left empty.*
- ***UNIQUE** prevents duplicate email addresses and vehicle numbers.*
- ***AUTO_INCREMENT** automatically generates unique ID values for primary keys.*

SQL Operations

*The basic **CRUD operations** were implemented using SQL.*

- ***INSERT:** Used to add new customer, vehicle, and service records to the database.*
- ***SELECT:** Used to retrieve existing records from the tables.*
- ***UPDATE:** Used to modify existing information, such as a customer's contact number or address.*
- ***DELETE:** Used to remove unwanted records from the database.*

These operations allow the service center to manage its database efficiently.

Data Retrieval

Different SQL clauses were used to retrieve and analyze specific information.

- ***WHERE**: Filters records based on a specified condition. For example, it can retrieve only services with Completed status.*
- ***ORDER BY**: Sorts records in ascending or descending order, such as sorting services by service cost.*
- ***LIKE**: Searches for records based on a partial value. For example, vehicle numbers beginning with TN37.*
- ***BETWEEN**: Retrieves records within a specified range, such as services with costs between ₹2,000 and ₹5,000.*
- ***GROUP BY**: Groups records based on a particular column, such as grouping services according to service type and calculating the number of services and total cost for each type.*

These operations make it easier to search, filter, sort, and analyze service center information.

Aggregate Functions and Joins

Aggregate functions are used to perform calculations on multiple records.

The database uses:

***COUNT()** – counts the total number of service records.*

***SUM()** – calculates the total service cost.*

***AVG()** – calculates the average service cost.*

***MAX()** – finds the highest service cost.*

***MIN()** – finds the lowest service cost.*

*An **INNER JOIN** was used to retrieve related information from multiple tables. The Customer, Vehicle, and Service tables are joined using their primary key and foreign key relationships. This allows the system to display information such as customer name, vehicle number, vehicle model, service type, service status, and service cost in a single result.*

6. Conclusion

Conclusion

The Vehicle Service Management Database was successfully developed using MySQL and SQL to manage customer, vehicle, and service information in a structured relational database. The system consists of three related tables: Customer, Vehicle, and Service, connected through primary and foreign keys.

The project implemented important database constraints such as PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, and AUTO_INCREMENT to maintain data integrity. Various SQL operations, including INSERT, SELECT, UPDATE, DELETE, WHERE, ORDER BY, LIKE, BETWEEN, aggregate functions, GROUP BY, and INNER JOIN, were used to manage and retrieve meaningful information from the database.

Through this project, the concepts of relational database design, table relationships, keys, constraints, data manipulation, data retrieval, aggregation, grouping, and joining tables were practically implemented using MySQL Workbench.

Future Enhancements

The database can be further improved by adding the following features:

- **Customer Login and Authentication:** Allow customers to securely log in and access their vehicle and service information.
- **Online Service Appointment Booking:** Enable customers to book service appointments online.
- **Automated Service Reminders:** Send reminders to customers when their vehicle service is due.
- **Service History Reports:** Generate detailed reports of previous services, costs, and service status.
- **Invoice Generation:** Automatically generate invoices based on service details and costs.
- **Mechanic Assignment and Management:** Add functionality to assign mechanics to specific service requests and manage their workload.
- **Web or Mobile Application Integration:** Connect the database with a web or mobile application for easier access and management.

- **Automated Database Backup and Recovery:** Implement regular backups and recovery mechanisms to prevent data loss.